



**UNIVERSITÀ
DEGLI STUDI
DI BERGAMO**

Dipartimento di
Ingegneria Gestionale, dell'Informazione e della Produzione

Corso di Laurea in
Ingegneria Informatica

Classe LM-8

Integrazione di eBPF in ambiente Android

Sfide e opportunità per l'ispezione di sistema

Candidato:

Mattia Ferrari

Matricola n. 1083721

Relatore:

Prof. Matteo Rossi

ANNO ACCADEMICO

2024/2025

Sommario

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat. Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Indice

1	Introduzione	1
1.1	Scenario Applicativo	1
1.2	Criticità	2
1.3	Approccio proposto	3
2	Tecnologie di base	5
2.1	Linux	5
2.2	Android Cuttlefish	6
2.3	Tracing, eBPF e bpftrace	8
2.3.1	Tracing delle applicazioni	8
2.3.2	Tracing mediante eBPF	9
2.3.3	bpftrace	10
3	Implementazione	15
3.1	Architettura generale	15
3.1.1	Le probe bpftrace	16
3.1.2	Il modulo di orchestrazione: monitor.py	17
3.1.3	Il modulo di analisi: summary.py	17
3.1.4	L'interfaccia utente: launcher.sh	17
3.2	Considerazioni progettuali	17
3.3	Le probe bpftrace	18
3.3.1	process_lifecycle.bt — Ciclo di vita dei processi	18
3.3.2	syscalls.bt — Monitoraggio delle system call	19
3.3.3	binder.bt — Monitoraggio IPC Binder	21

3.3.4	Probe di rete: net_send, net_recv, net_bind, net_accept . .	22
3.4	L'orchestratore: monitor.py	24
3.4.1	Gestione del processo bpfttrace	24
3.4.2	Elaborazione degli eventi	25
3.4.3	Gestione della sessione e terminazione	26
3.5	Il launcher interattivo: launcher.sh	26
3.5.1	Modalità operative	26
3.6	Il modulo di analisi: summary.py	27
3.6.1	Analisi generale degli eventi	27
3.6.2	Analisi delle syscall e della latenza	28
3.6.3	Analisi del Binder IPC	28
3.6.4	Analisi di rete	28
3.6.5	Albero dei processi e resource map	29
3.6.6	Output	29
3.7	Il formato dei dati: JSON Lines	29
4	Test e Validazione	35
4.1	Attivazione della modalità aereo	35
4.2	Navigazione web con Chrome	38
4.3	Apertura della galleria e acquisizione fotografica	42
5	Limiti e possibili miglioramenti	49
6	Conclusioni	51
	Bibliografia	53

Capitolo 1

Introduzione

1.1 Scenario Applicativo

Negli ultimi anni i dispositivi mobili hanno assunto un ruolo centrale nell'ambito dell'informatica personale e professionale. Tra i sistemi operativi per dispositivi mobili, Android rappresenta una delle piattaforme più diffuse a livello mondiale [16] e viene utilizzato in una grande varietà di dispositivi, dagli smartphone ai sistemi embedded. La complessità crescente delle applicazioni e del sistema operativo rende sempre più importante disporre di strumenti in grado di analizzare e comprendere il comportamento delle applicazioni durante l'esecuzione.

Il monitoraggio delle attività delle applicazioni è fondamentale in diversi ambiti, tra cui il debugging del software, l'analisi delle prestazioni e la sicurezza informatica. In particolare, la possibilità di osservare le interazioni tra le applicazioni e il sistema operativo consente di individuare anomalie di funzionamento, colli di bottiglia prestazionali e potenziali comportamenti malevoli.

Tradizionalmente il tracing del sistema operativo viene realizzato mediante strumenti specifici che permettono di raccogliere informazioni sull'esecuzione dei processi e sulle attività del kernel. Tali strumenti possono tuttavia introdurre un significativo overhead computazionale oppure richiedere modifiche al sistema operativo, rendendone difficile l'utilizzo in ambienti reali. Inoltre, molti strumenti di analisi disponibili nei sistemi operativi tradizionali, e in particolare in ambiente Linux, non sono direttamente utilizzabili su Android senza adattamenti specifici.

Dal 2014, con l'introduzione dell'extended Berkeley Packet Filter (eBPF) in Linux [9], è stata resa disponibile una tecnologia che permette di eseguire piccoli programmi in modo sicuro ed efficiente all'interno del sistema operativo. Questo consente di osservare il comportamento delle applicazioni e del sistema senza la necessità di modifiche al codice sorgente o dello sviluppo di moduli dedicati. Per queste ragioni eBPF è diventato uno strumento sempre più utilizzato per attività di tracing e osservabilità nei sistemi Linux [8].

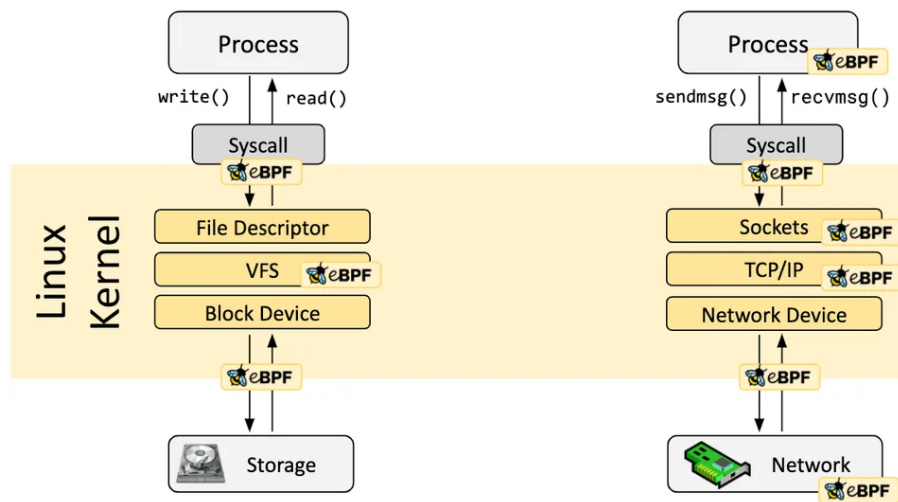


Figura 1.1: Architettura di eBPF nel kernel Linux

1.2 Criticità

Tra gli strumenti che permettono di utilizzare eBPF in modo pratico, `bpftool` costituisce una soluzione particolarmente efficace per il tracing delle applicazioni e del sistema operativo. `bpftool` [5] mette a disposizione un linguaggio di scripting ad alto livello che consente di definire sonde e azioni associate agli eventi osservati, permettendo di raccogliere informazioni durante l'esecuzione senza sviluppare programmi eBPF completi. Questo approccio semplifica notevolmente la realizzazione di strumenti di analisi dinamica.

Tuttavia, l'utilizzo di `bpftool` in ambiente Android presenta diverse difficoltà pratiche. In molti dispositivi l'accesso alle funzionalità di basso livello del sistema è limitato e spesso non sono disponibili gli strumenti e le librerie necessari per

l'esecuzione di programmi basati su eBPF. Inoltre, l'installazione diretta di tali strumenti sul sistema Android può risultare complessa o non praticabile, soprattutto in ambienti di produzione, dove tipicamente non sono disponibili i permessi di root necessari all'installazione e all'utilizzo di tali strumenti. Ottenere tali permessi richiederebbe di effettuare il rooting del dispositivo o di sostituire la release Android installata, operazioni spesso non praticabili in contesti reali.

	Emulatore	Dispositivo di Produzione
Accesso root	Disponibile	Richiede rooting
Installazione strumenti	Possibile	Richiede rooting
Modifica della ROM	Non necessaria	Spesso necessaria
Utilizzo di bpftrace	Fattibile	Non praticabile
Riproducibilità	Alta	Bassa

Figura 1.2: Confronto tra ambiente di test e dispositivo di produzione Android.

Queste limitazioni rendono difficile l'impiego delle tecnologie basate su eBPF per l'analisi del comportamento delle applicazioni Android, nonostante il loro potenziale per il monitoring del sistema. Risulta quindi necessario definire un ambiente di lavoro che consenta l'utilizzo di bpftrace in modo stabile e riproducibile, permettendo allo stesso tempo l'osservazione delle attività delle applicazioni durante l'esecuzione.

Il problema affrontato in questa tesi consiste quindi nel rendere possibile l'utilizzo di bpftrace per il tracing delle applicazioni in ambiente Android, individuando una configurazione che consenta di superare le limitazioni tipiche del sistema e di ottenere dati significativi sull'esecuzione delle applicazioni.

1.3 Approccio proposto

Per superare le difficoltà legate all'utilizzo degli strumenti basati su eBPF in ambiente Android, in questo lavoro è stata sviluppata una soluzione che permette di eseguire lo strumento bpftrace in un ambiente controllato e riproducibile, mantenendo

do allo stesso tempo la possibilità di osservare il comportamento delle applicazioni Android.

La soluzione proposta si basa sulla creazione di un ambiente di sviluppo virtualizzato in cui sia possibile eseguire Android e utilizzare strumenti tipicamente disponibili nei sistemi Linux tradizionali. A questo scopo è stato utilizzato l'emulatore Android Cuttlefish [2], eseguito su sistema operativo Ubuntu, che consente di disporre di un ambiente Android completo e configurabile per attività di sviluppo e sperimentazione.

All'interno dell'ambiente Android è stata installata una distribuzione Linux minimale basata su Debian. Questa distribuzione è stata utilizzata come ambiente di lavoro per l'esecuzione di bpfttrace e degli strumenti necessari al tracing. L'utilizzo di una distribuzione Linux separata ha permesso di evitare la compilazione nativa di bpfttrace su Android e di semplificare l'installazione delle dipendenze necessarie.

In questo modo è stato possibile realizzare un ambiente che combina la flessibilità degli strumenti Linux con la possibilità di osservare il comportamento di applicazioni eseguite su Android. Su questa base è stato sviluppato un insieme di script e strumenti software che permettono di eseguire sessioni di tracing e di raccogliere automaticamente i dati prodotti da bpfttrace.

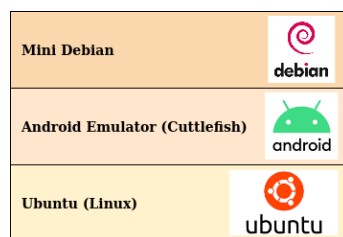


Figura 1.3: Architettura della soluzione proposta: Ubuntu, Cuttlefish e ambiente Debian.

Capitolo 2

Tecnologie di base

2.1 Linux

Linux è un sistema operativo di tipo Unix-like ampiamente utilizzato sia in ambito accademico sia industriale [**linux_kernel**] grazie alla sua stabilità, flessibilità e natura open source. Il sistema è basato su un'architettura in cui il kernel gestisce le risorse hardware e fornisce ai programmi un'interfaccia standardizzata attraverso le system call, mentre l'insieme dei programmi in spazio utente costituisce l'ambiente operativo vero e proprio. Questa organizzazione rende Linux particolarmente adatto allo sviluppo software e alle attività di sperimentazione, poiché consente un controllo dettagliato del sistema e un facile accesso agli strumenti di analisi e debugging.

Nel presente lavoro è stato utilizzato come sistema operativo di base Ubuntu 22.04 LTS, una distribuzione Linux ampiamente diffusa e supportata a lungo termine. Ubuntu è stata scelta per la disponibilità di pacchetti software aggiornati e per la semplicità di configurazione dell'ambiente di sviluppo. Il sistema Ubuntu ha costituito la piattaforma principale su cui sono stati installati gli strumenti necessari allo svolgimento della tesi, tra cui l'ambiente di virtualizzazione Android Cuttlefish e i componenti utilizzati per il tracing basato su eBPF. L'utilizzo di una distribuzione Linux standard ha permesso di disporre di un ambiente stabile e riproducibile, facilitando l'installazione delle dipendenze e la gestione degli strumenti software impiegati nelle sperimentazioni.

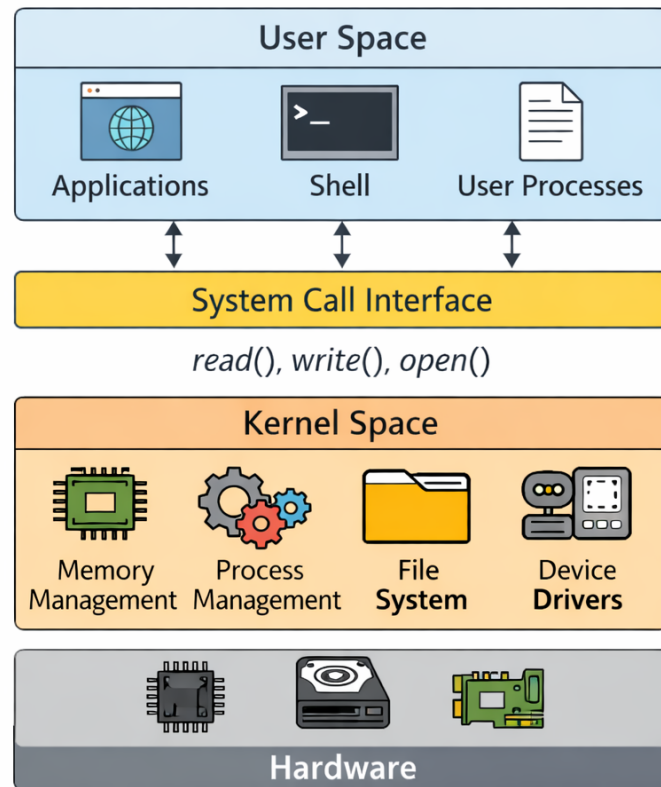


Figura 2.1: Architettura del sistema Linux: kernel space, user space e system call.

2.2 Android Cuttlefish

Android [**android_aosp**] è un sistema operativo per dispositivi mobili basato sul kernel Linux e progettato per supportare l'esecuzione di applicazioni in un ambiente isolato e controllato. Il sistema è organizzato in una struttura a livelli che comprende il kernel Linux, le librerie di sistema, il runtime Android, il framework delle applicazioni e il livello delle applicazioni utente. Il kernel Linux costituisce il livello più basso dell'architettura e si occupa della gestione delle risorse hardware, della memoria, dei processi, dei dispositivi di input/output e delle comunicazioni di rete. I livelli superiori forniscono invece i servizi necessari all'esecuzione delle applicazioni e definiscono l'ambiente software tipico della piattaforma Android.

Ogni applicazione Android viene eseguita in uno spazio isolato rispetto alle altre applicazioni, generalmente associato a un identificatore utente distinto. Questo

meccanismo di isolamento contribuisce alla sicurezza del sistema ma limita l'accesso diretto alle risorse e alle funzionalità di basso livello. Inoltre, molte componenti tipiche delle distribuzioni Linux tradizionali non sono presenti oppure risultano fortemente ridotte, rendendo più complesso l'utilizzo di strumenti di analisi progettati per ambienti Linux standard.

Nonostante Android utilizzi il kernel Linux, l'ambiente software differisce in modo significativo da quello di una distribuzione Linux tradizionale. In particolare, l'organizzazione del file system, la disponibilità delle librerie e la gestione dei permessi rendono spesso necessario un adattamento degli strumenti sviluppati per sistemi Linux convenzionali. Questo aspetto è particolarmente rilevante nel caso degli strumenti di tracing, che richiedono generalmente l'accesso a funzionalità di basso livello del sistema operativo.



Figura 2.2: Architettura a livelli del sistema operativo Android.

Per lo sviluppo e la sperimentazione è stato utilizzato Android Cuttlefish [2], un

emulatore ufficiale della piattaforma Android progettato per l'esecuzione su sistemi Linux. Cuttlefish permette di eseguire una o più istanze complete del sistema operativo Android all'interno di un ambiente virtualizzato, consentendo di simulare il comportamento di un dispositivo reale senza la necessità di utilizzare hardware fisico.

Cuttlefish utilizza tecnologie di virtualizzazione disponibili in ambiente Linux per eseguire il sistema Android in una macchina virtuale che riproduce i principali componenti di un dispositivo reale. L'ambiente virtualizzato include il kernel Android, il sistema operativo e i servizi necessari all'esecuzione delle applicazioni. Questo approccio permette di ottenere un ambiente di test controllato e riproducibile, caratteristica particolarmente importante nelle attività di sviluppo e sperimentazione.

Nel presente lavoro Cuttlefish è stato utilizzato come ambiente di esecuzione del sistema Android su cui effettuare le attività di tracing. L'emulatore è stato eseguito su sistema operativo Ubuntu, che ha costituito la piattaforma principale per l'installazione e la configurazione degli strumenti necessari allo svolgimento della tesi. L'utilizzo di un ambiente virtualizzato ha permesso di effettuare le sperimentazioni in modo riproducibile e di controllare la configurazione del sistema senza intervenire su dispositivi fisici.

2.3 Tracing, eBPF e bpftrace

2.3.1 Tracing delle applicazioni

Il tracing delle applicazioni è una tecnica di analisi che consiste nella raccolta sistematica di informazioni durante l'esecuzione di un programma o di un sistema operativo, con l'obiettivo di osservare il comportamento dinamico del software e le sue interazioni con il sistema. A differenza dell'analisi statica, che si basa sull'esame del codice sorgente o dei file eseguibili, il tracing consente di studiare ciò che avviene realmente durante l'esecuzione, permettendo di ottenere informazioni temporali sugli eventi generati dal sistema.

Nel contesto dei sistemi operativi moderni, il tracing può essere effettuato a diversi livelli di osservazione. A livello applicativo è possibile monitorare le operazioni effettuate da un processo, come la creazione di nuovi processi, l'accesso ai file o le comunicazioni di rete. A livello di sistema operativo è invece possibile osservare eventi gestiti dal kernel, come le chiamate di sistema, la schedulazione dei processi o le operazioni di input/output. Questa possibilità di osservare eventi a diversi livelli rende il tracing uno strumento fondamentale per comprendere il funzionamento complessivo di un sistema software.

Le tecniche di tracing vengono utilizzate principalmente per tre scopi. Il primo è il debugging del software, dove l'osservazione dettagliata delle operazioni eseguite da un programma permette di individuare errori o comportamenti inattesi. Il secondo è l'analisi delle prestazioni, in cui il tracing consente di identificare colli di bottiglia e di comprendere come le risorse del sistema vengono utilizzate dalle applicazioni. Il terzo ambito è quello della sicurezza informatica, dove l'osservazione delle attività dei processi permette di individuare comportamenti anomali o potenzialmente malevoli.

Tradizionalmente il tracing in ambiente Linux è stato realizzato tramite strumenti basati su meccanismi come `ptrace` [13], utilizzati ad esempio da programmi come `strace` [17], oppure tramite infrastrutture di tracing integrate nel kernel come `ftrace` [15] e `perf` [12]. Questi strumenti permettono di ottenere informazioni dettagliate sul comportamento del sistema, ma possono risultare complessi da utilizzare oppure introdurre un overhead significativo durante l'esecuzione.

2.3.2 Tracing mediante eBPF

Una delle tecnologie più recenti per il tracing nei sistemi Linux è rappresentata dall'extended Berkeley Packet Filter (eBPF) [9]. eBPF permette di eseguire piccoli programmi all'interno del sistema operativo in modo controllato e sicuro, consentendo di intercettare eventi durante l'esecuzione senza modificare il codice del kernel o delle applicazioni.

Il funzionamento di eBPF si basa sull'inserimento dinamico di programmi che vengono eseguiti quando si verificano determinati eventi, come l'ingresso in una funzione del kernel o l'attivazione di un tracepoint. Prima di essere caricati, i pro-

programmi eBPF vengono verificati da un componente del sistema operativo chiamato verifier, che ne controlla la sicurezza e garantisce che non possano compromettere la stabilità del sistema. Questo meccanismo permette di eseguire codice in modo sicuro anche all'interno di parti critiche del sistema operativo.

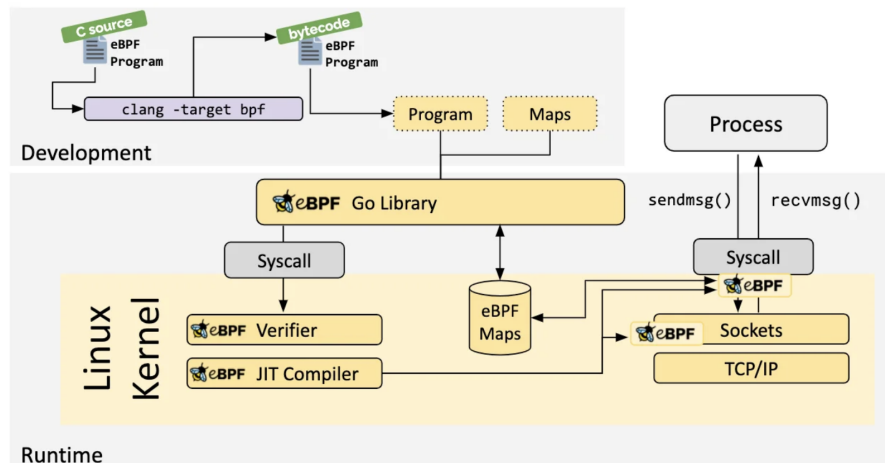


Figura 2.3: Flusso di caricamento ed esecuzione di un programma eBPF nel kernel Linux.

Rispetto alle tecniche di tracing tradizionali, eBPF presenta alcune caratteristiche distintive. In primo luogo consente di raccogliere informazioni con un overhead generalmente ridotto, poiché i programmi vengono eseguiti direttamente nel contesto dell'evento osservato. Inoltre, le sonde possono essere attivate e disattivate dinamicamente senza riavviare il sistema operativo. Un ulteriore vantaggio è la grande flessibilità, che permette di definire nuove sonde e nuovi tipi di analisi senza modificare il sistema.

Grazie a queste caratteristiche eBPF rappresenta oggi una tecnologia particolarmente adatta per il tracing dinamico dei sistemi Linux e costituisce la base di numerosi strumenti moderni di osservabilità.

2.3.3 bpftrace

Per utilizzare eBPF in modo pratico sono stati sviluppati diversi strumenti software. Tra questi, **bpftrace** [5] rappresenta uno degli strumenti più diffusi per la realizzazione di script di tracing basati su eBPF.

bpfttrace è un linguaggio di scripting e un interprete che permette di definire programmi di tracing in forma compatta e leggibile. Gli script vengono tradotti automaticamente in programmi eBPF ed eseguiti dal sistema operativo, semplificando notevolmente lo sviluppo rispetto alla scrittura diretta di codice eBPF.

Un programma **bpfttrace** è costituito da una o più probe, cioè punti di osservazione associati a eventi specifici. Ogni probe è composta da due parti principali:

- la definizione dell'evento da osservare
- le azioni da eseguire quando l'evento si verifica

La struttura generale di una probe è illustrata in Figura 2.4, che evidenzia la separazione tra la definizione dell'evento da osservare e il blocco delle azioni da eseguire.

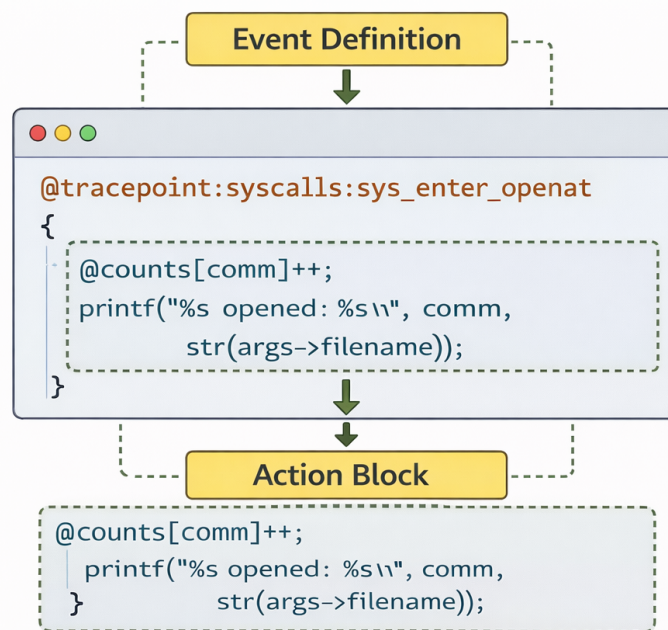


Figura 2.4: Struttura di una probe bpfttrace: definizione dell'evento e blocco delle azioni.

bpfttrace supporta diversi tipi di probe, che permettono di osservare eventi a diversi livelli del sistema. Nel presente lavoro sono stati utilizzati esclusivamente i

tracepoint, in quanto rappresentano la soluzione più stabile e compatibile tra diverse versioni del kernel.

I tracepoint sono punti di osservazione statici definiti all'interno del sistema operativo. A differenza di altre tipologie di probe, la loro interfaccia è stabile tra diverse versioni del kernel, il che li rende particolarmente adatti a un ambiente come Android, dove la versione del kernel può variare. Ogni tracepoint espone un insieme strutturato di argomenti che descrivono l'evento osservato, consentendo di accedere direttamente a informazioni quali il processo coinvolto, i parametri della chiamata e il risultato dell'operazione. I tracepoint disponibili nel sistema utilizzato durante le sperimentazioni sono illustrati in Figura 2.5.

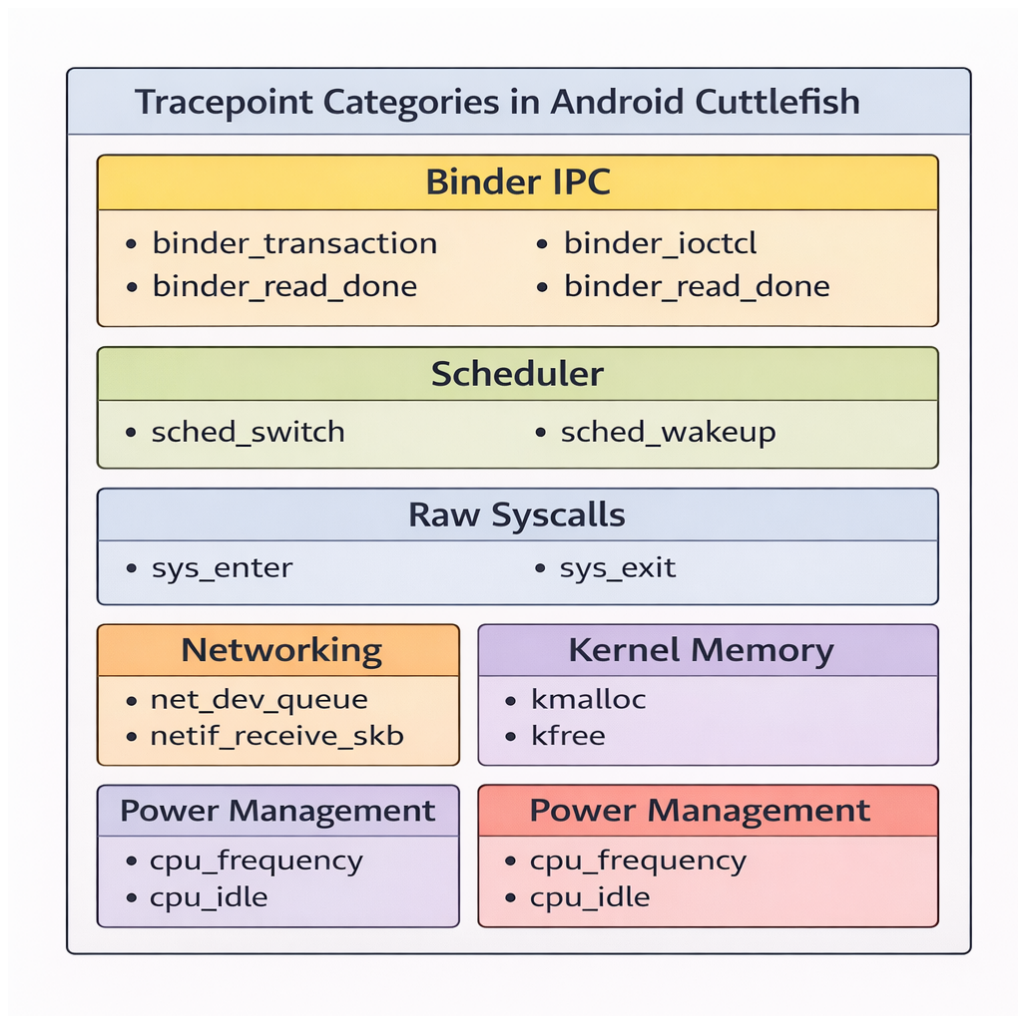


Figura 2.5: Lista dei tracepoint disponibili nell'ambiente Android Cuttlefish utilizzato nelle sperimentazioni.

`bpfttrace` supporta anche altri due tipi di probe. I `kprobe` consentono di inserire sonde dinamiche all'ingresso o all'uscita di funzioni del kernel, offrendo grande flessibilità anche in punti privi di tracepoint predefiniti; la loro stabilità è tuttavia inferiore, poiché dipendono dai nomi interni delle funzioni del kernel che possono variare tra versioni diverse. Le `uprobe` operano invece in spazio utente e permettono di tracciare funzioni di librerie o applicazioni specifiche senza modificarne il codice, risultando utili quando l'obiettivo è osservare il comportamento interno di un singolo processo piuttosto che eventi di sistema.

Capitolo 3

Implementazione

3.1 Architettura generale

Il sistema è composto da quattro componenti principali che collaborano secondo un'architettura a pipeline: le probe `bpftrace` raccolgono eventi direttamente dal kernel, `monitor.py` li normalizza e persiste, `summary.py` li analizza in fase post-sessione, e `launcher.sh` orchestra l'intera interazione con l'utente. L'architettura complessiva è illustrata in Figura 3.1.

Il flusso di dati segue un percorso ben definito: le probe producono eventi strutturati in formato JSON su stdout, `monitor.py` li consuma riga per riga e li persiste in un file `.jsonl`, e `summary.py` legge questo file al termine della sessione per produrre report e statistiche aggregate. `launcher.sh` funge da punto di ingresso unico che coordina l'avvio delle probe, la durata della sessione e la generazione del report finale.

Questa separazione delle responsabilità risponde a un principio architetturale preciso: ciascun componente è progettato per fare una sola cosa e farlo bene, rimanendo testabile e utilizzabile in modo indipendente. Le interfacce di comunicazione tra i componenti sono deliberatamente semplici — stdout/stdin per il flusso di eventi, file JSON per la configurazione e la persistenza — in modo da minimizzare le dipendenze e facilitare la manutenzione.

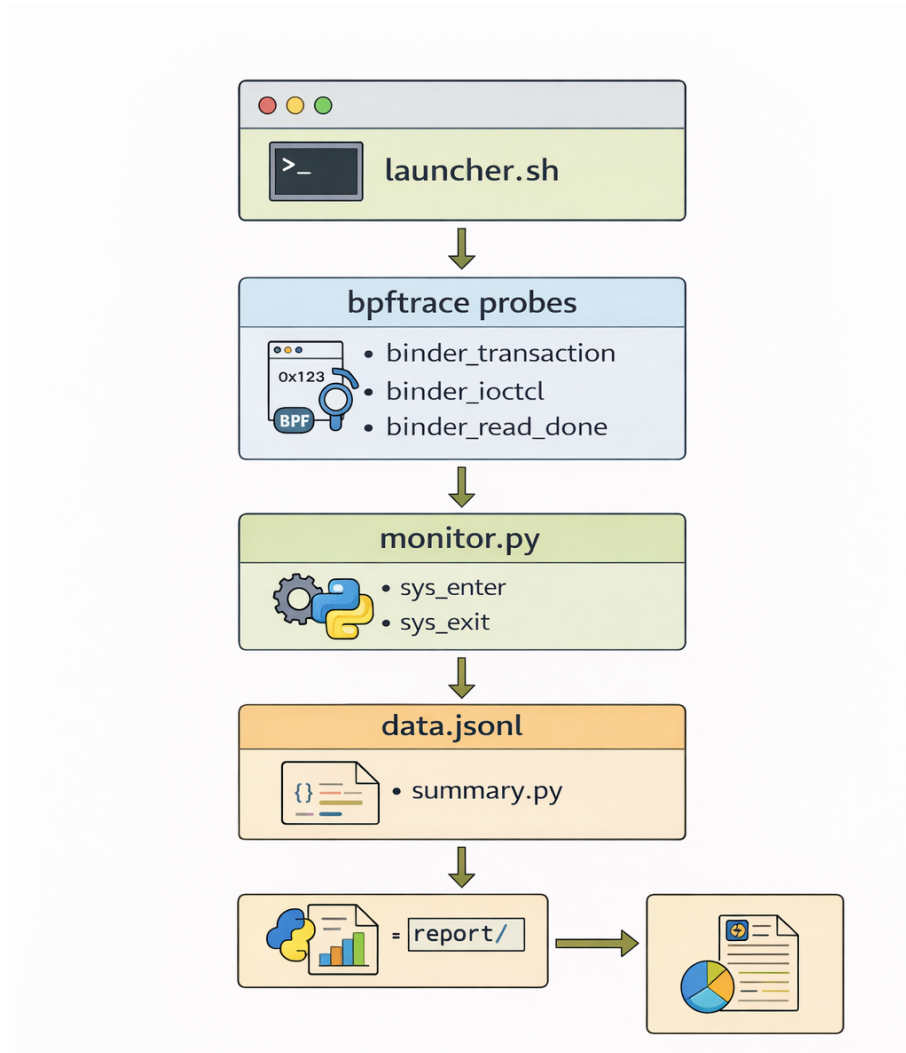


Figura 3.1: Architettura del sistema: flusso di dati tra probe bpfttrace, monitor.py, summary.py e launcher.sh.

3.1.1 Le probe bpfttrace

Le probe operano interamente in spazio kernel, con accesso diretto alle strutture dati del kernel quali `task_struct` [18], e producono output in formato JSON su stdout. La scelta di JSON come formato di output permette a bpfttrace di comportarsi come un produttore di eventi strutturati che il livello superiore può consumare senza logica di parsing ad hoc. Ogni evento emesso segue uno schema comune con i campi: `ts_ns` (timestamp in nanosecondi), `ts` (orario leggibile), `type` (categoria dell'evento), `event` (nome specifico dell'evento), `pid`, `ppid`, `tid`, `uid`, `comm` (nome del processo, massimo 15 caratteri imposto dal kernel) e un oggetto `data` contenente

i campi specifici della probe.

3.1.2 Il modulo di orchestrazione: `monitor.py`

`monitor.py` è il cuore del sistema. Si occupa di avviare `bpfttrace` come sottoprocesso, gestire il ciclo di vita della sessione di raccolta dati, normalizzare e validare gli eventi ricevuti, arricchirli dove necessario tramite il filesystem `/proc` e salvarli su disco in formato JSON Lines (`.jsonl`). Opera come consumer dello stream stdout di `bpfttrace`, consumando gli eventi riga per riga in modo sincrono, mentre un thread dedicato drena contemporaneamente stderr per separare gli errori diagnostici.

3.1.3 Il modulo di analisi: `summary.py`

`summary.py` è il modulo di analisi post-sessione. Legge il file `events.jsonl` prodotto da `monitor.py`, calcola statistiche aggregate su syscall, processi, Binder IPC e attività di rete, e genera un report testuale sia in formato plain text che Markdown. Il modulo produce inoltre un file `index.json` all'interno della directory di sessione, contenente tutti i dati analitici in forma strutturata.

3.1.4 L'interfaccia utente: `launcher.sh`

`launcher.sh` è lo script bash che funge da interfaccia utente per il sistema. Offre un menu interattivo a colori che permette di scegliere quale probe eseguire, per quanto tempo, e se generare automaticamente un report al termine della sessione. Supporta anche una modalità non interattiva tramite flag da riga di comando (`-p` per specificare la probe, `-t` per il timer).

3.2 Considerazioni progettuali

Alcune scelte progettuali meritano una riflessione esplicita. La separazione tra la probe `bpfttrace` (che produce dati grezzi) e `monitor.py` (che normalizza, arricchisce e persiste) risponde a un principio di separazione delle responsabilità: `bpfttrace` è ottimizzato per la raccolta ad alta frequenza di eventi in spazio kernel, mentre

Python è più adatto per la logica applicativa complessa come la lettura di `/proc` o la gestione delle sessioni. Un'unica componente che facesse entrambe le cose sarebbe più fragile e difficile da mantenere.

La scelta di correlare `sys_enter` e `sys_exit` nelle probe di rete (invece di usare solo `sys_enter`) aggiunge la dimensione temporale all'osservazione: non solo si sa che una syscall è stata invocata, ma anche quanto ha impiegato e se ha avuto successo. Questa informazione è preziosa per l'analisi delle prestazioni e per il rilevamento di anomalie basato su latenza.

La suddivisione dell'attività di rete in quattro probe separate (invece di una sola probe che monitora tutte le syscall di rete) è una scelta di modularità che permette all'utente di attivare solo il sottoinsieme di informazioni di cui ha bisogno, riducendo il volume di dati prodotti e l'overhead sul sistema monitorato.

Infine, la presenza di `launcher.sh` come wrapper bash — invece di integrare la gestione del timer direttamente in `monitor.py` — riflette la filosofia Unix di strumenti semplici e componibili [14]: `monitor.py` si occupa solo di monitorare, il launcher si occupa dell'orchestrazione e dell'esperienza utente, e `summary.py` si occupa solo dell'analisi. Questa separazione rende ciascun componente testabile e utilizzabile indipendentemente.

3.3 Le probe bpftrace

Il progetto include sette probe bpftrace, ognuna dedicata a osservare una specifica classe di eventi di sistema. La directory `probes/` contiene tutti gli script `.bt`, mentre la loro configurazione — metadati, codice identificativo, tipo di evento atteso — è centralizzata nel file `config/probes_map.json`, che funge da registro consultato sia da `monitor.py` che da `launcher.sh`.

3.3.1 `process_lifecycle.bt` — Ciclo di vita dei processi

Questa probe monitora le tre fasi fondamentali del ciclo di vita di un processo: la creazione (*fork*), l'esecuzione di un nuovo programma (*exec*) e la termina-

zione (*exit*). Per farlo, aggancia tre tracepoint dello scheduler del kernel Linux: `sched:sched_process_fork`, `sched:sched_process_exec` e `sched:sched_process_exit`.

La scelta di questi tracepoint è motivata dalla loro stabilità e dalla ricchezza di informazioni che espongono. I tracepoint `sched_process_*` sono presenti e stabili in tutte le versioni del kernel Linux utilizzate da Android, e forniscono direttamente i metadati del processo coinvolto senza necessità di accedere a strutture kernel aggiuntive. Rispetto all'alternativa di intercettare le syscall `fork` o `execve` tramite `raw_syscalls`, questi tracepoint vengono attivati in un momento del ciclo di vita del processo in cui le strutture dati del kernel sono già aggiornate e consistenti, riducendo il rischio di osservare stati intermedi.

Un aspetto rilevante è la raccolta del `ppid` (Process ID del processo padre), ottenuto tramite un cast esplicito alla struttura kernel `task_struct`: il campo `real_parent->tgid` fornisce l'identificativo del gruppo di thread del processo genitore, informazione fondamentale per ricostruire l'albero genealogico dei processi. Il campo `comm`, recuperato direttamente dal kernel, è soggetto al limite di 15 caratteri imposto dal kernel Linux; per questa ragione, `monitor.py` arricchisce gli eventi di tipo `exec` leggendo il file `/proc/<pid>/cmdline` prima che il processo termini, ottenendo così la riga di comando completa.

Gli eventi generati da questa probe sono utilizzati da `summary.py` per costruire l'albero dei processi e correlare l'attività dei diversi componenti del sistema Android.

3.3.2 `syscalls.bt` — Monitoraggio delle system call

La probe `syscalls.bt` utilizza il tracepoint generico `raw_syscalls:sys_enter`, che viene attivato all'ingresso di *qualsiasi* system call invocata nel sistema. Per limitare l'osservazione alle sole syscall di interesse, la probe applica un filtro inline sul campo `args->id`, che contiene il numero identificativo della syscall:

- `execve` (id 59): esecuzione di un nuovo programma;
- `openat` (id 257): apertura di un file;
- `connect` (id 42): tentativo di connessione di rete.

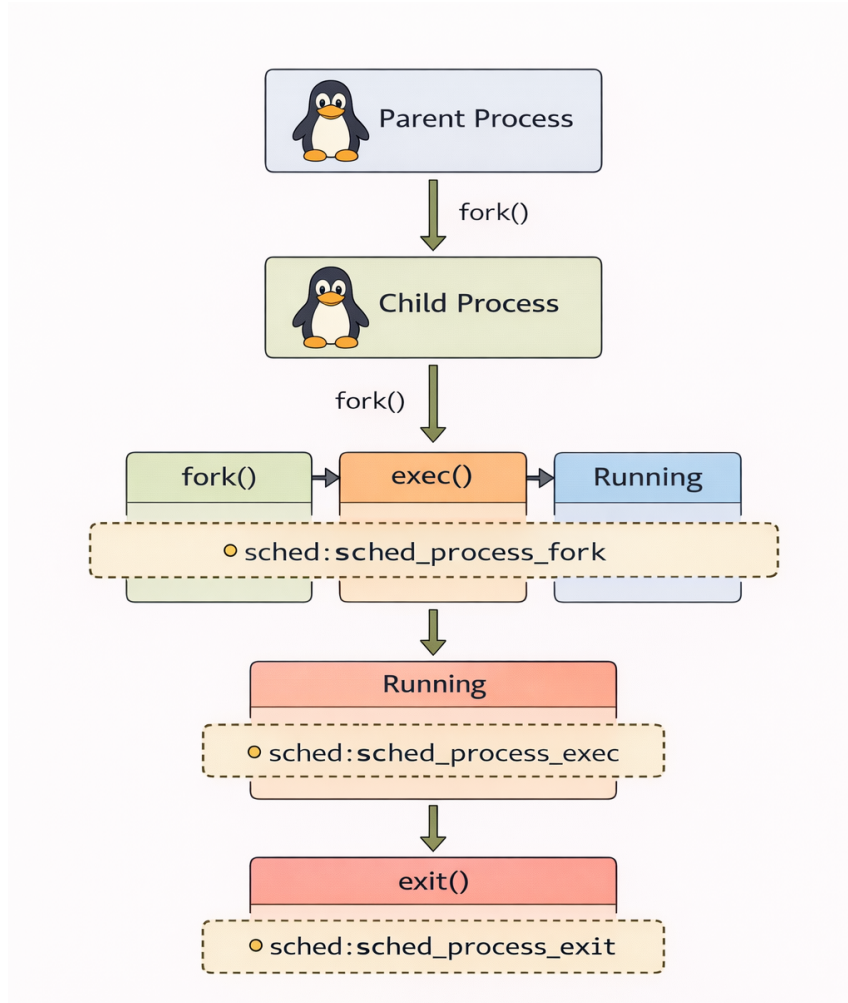


Figura 3.2: Diagramma del ciclo di vita di un processo Linux: fork, exec e exit con i relativi tracepoint osservati.

Questo approccio — intercettare tutto e filtrare — è preferibile rispetto all'uso di tracepoint specifici per syscall (come `tracepoint:syscalls:sys_enter_execve`) perché consente di gestire tutte le syscall di interesse con un unico handler, riducendo la duplicazione del codice e semplificando la gestione delle mappe temporanee condivise.

La probe implementa una decodifica semantica degli argomenti per ciascuna syscall. Per `execve` viene letto il percorso del binario da eseguire; per `openat` il percorso del file da aprire; per `connect` viene ispezionata la struttura `sockaddr` per determinare la famiglia di indirizzi (`AF_UNIX`, `AF_INET` o `AF_INET6`) e decodificare l'indirizzo di destinazione.

Un aspetto critico da considerare nella lettura di questi parametri è il problema del *time-of-check to time-of-use* (TOCTOU) [4]. I percorsi dei file letti da `execve` e `openat`, così come gli indirizzi letti dalla struttura `sockaddr` in `connect`, risiedono nella memoria del processo utente. Nel momento in cui la probe li legge tramite `uptr()`, il processo potrebbe in teoria modificarli prima che il kernel li utilizzi effettivamente. Questo significa che il valore osservato dalla probe potrebbe non corrispondere al valore effettivamente usato dal kernel per eseguire l'operazione. Per scopi di tracing comportamentale e analisi forense questo margine di incertezza è generalmente accettabile, ma è importante tenerne conto quando si interpretano i dati raccolti, in particolare in contesti di sicurezza dove un attaccante potrebbe sfruttare questa finestra temporale.

Un aspetto tecnico degno di nota riguarda la gestione di `AF_INET`: la funzione `ntop()` di `bpftool`, necessaria per convertire un indirizzo IPv4 in formato testuale, non può essere assegnata a una variabile ma può essere utilizzata solo inline in un'istruzione `printf`. Per questo motivo la probe gestisce il caso `AF_INET` in un ramo separato del codice, emettendo l'evento con una `printf` dedicata che include `ntop()` direttamente nell'argomento stringa di formato, e ritorna immediatamente con `return` senza passare dal `printf` generico.

Per evitare memory leak nelle mappe `bpftool` utilizzate come variabili temporanee indicizzate per `tid`, la probe include un handler del tracepoint `sched:sched_process_exit` che cancella i valori residui, e un handler `END` che pulisce le mappe al termine.

3.3.3 binder.bt — Monitoraggio IPC Binder

Binder [1] è il meccanismo di Inter-Process Communication (IPC) fondamentale di Android. Ogni chiamata a un servizio di sistema, ogni interazione tra applicazioni e framework Android, ogni chiamata a un Content Provider passa attraverso Binder. Monitorare Binder significa quindi avere visibilità sul tessuto connettivo dell'intero sistema operativo Android.

La scelta di usare i tracepoint del driver Binder è obbligata: non esistono syscall dirette che esponano le transazioni Binder in modo strutturato, poiché tutta la comunicazione avviene tramite il driver `/dev/binder` attraverso `ioctl`. I trace-

point `binder:binder_transaction_*` sono l'unico punto di osservazione che fornisce informazioni strutturate sulle transazioni a livello kernel, senza necessità di intercettare e decodificare le `ioctl` manualmente.

La probe aggancia tre tracepoint esposti dal driver kernel del Binder:

- `binder:binder_transaction`: generato all'avvio di una transazione, cattura mittente, destinatario e metadati della chiamata;
- `binder:binder_transaction_received`: generato quando il destinatario riceve la transazione, permette di correlare invio e ricezione tramite il campo `debug_id`;
- `binder:binder_transaction_alloc_buf`: generato quando viene allocato il buffer dati, fornisce la dimensione del payload trasferito.

Il campo `flags` viene decodificato per determinare se la transazione è one-way (asincrona, bit 0x01 dei flag impostato) o sincrona. Il `ppid` è recuperato tramite cast a `task_struct` come nelle altre probe.

La correlazione tra i tre eventi tramite `debug_id` è fondamentale: solo incrociando `binder_transaction` (che ha i dati del mittente), `binder_transaction_alloc_buf` (che ha la dimensione del payload) e `binder_transaction_received` (che ha i dati del destinatario) è possibile ricostruire un'immagine completa di ogni transazione IPC. Questa correlazione viene eseguita da `summary.py` nella fase di analisi.

3.3.4 Probe di rete: `net__send`, `net__recv`, `net__bind`, `net__accept`

L'attività di rete è monitorata tramite quattro probe specializzate, ciascuna dedicata a una syscall diversa. La scelta di osservare le syscall di rete tramite il tracepoint generico `raw_syscalls:sys_enter/sys_exit` — filtrato per numero di syscall — invece di tracepoint specifici è motivata dalla necessità di correlare ingresso e uscita della stessa syscall per calcolare la latenza e leggere il valore di ritorno. Questa tecnica di correlazione richiede di salvare il timestamp e il contesto all'ingresso in una mappa indicizzata per `tid`, e di recuperarli all'uscita. La suddivisione in probe separate è inoltre una scelta progettuale che offre flessibilità: è possibile attivare solo

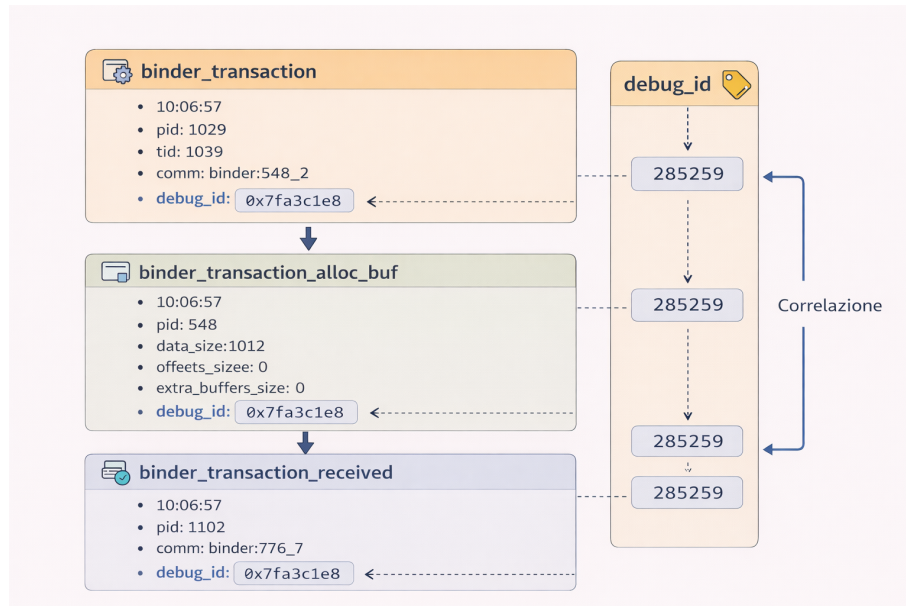


Figura 3.3: Schema della correlazione tra i tre tracepoint Binder tramite il campo `debug_id`.

il monitoraggio delle connessioni in uscita, o solo il volume di dati inviati, o solo i servizi in ascolto, senza attivare l'intero set.

Anche per le probe di rete vale la considerazione TOCTOU già discussa per `syscalls.bt`: gli indirizzi letti dalla struttura `sockaddr` in memoria utente potrebbero in teoria essere modificati dal processo tra il momento della lettura da parte della probe e il momento in cui il kernel utilizza effettivamente il valore.

net_send.bt

Traccia la syscall `sendto` (id 44). Utilizza la tecnica di correlazione `sys_enter/sys_exit`: all'ingresso della syscall vengono salvate in mappe bpftrace indicizzate per `tid` le informazioni di contesto (timestamp, ppid, file descriptor, dimensione richiesta); all'uscita viene calcolata la latenza come differenza di timestamp in nanosecondi e convertita in microsecondi. Il valore di ritorno `args->ret` corrisponde al numero di byte effettivamente inviati, che può differire dalla dimensione richiesta in caso di invio parziale.

net_recv.bt

Funziona in modo simmetrico rispetto a `net_send.bt` ma aggancia `recvfrom` (id 45). Registra la dimensione del buffer allocato per la ricezione e il numero di byte

effettivamente ricevuti, oltre alla latenza della syscall.

net_bind.bt

Traccia `bind` (id 49), la syscall con cui un processo richiede al kernel di associare un socket a un indirizzo locale, tipicamente per mettersi in ascolto su una porta. La probe decodifica la struttura `sockaddr` all'ingresso della syscall, distinguendo `AF_INET` (indirizzi IPv4, con decodifica di porta e indirizzo tramite `ntop()`) e `AF_UNIX` (socket locali, con lettura del percorso). Solo i bind andati a buon fine (valore di ritorno uguale a 0) vengono evidenziati nell'analisi come porte effettivamente aperte.

net_accept.bt

Traccia `accept` (id 43) e `accept4` (id 288), le syscall con cui un processo accetta una connessione in ingresso. Il peer address viene letto dalla struttura `sockaddr` popolata dal kernel all'uscita della syscall, quando il descrittore della nuova connessione è già disponibile. In fase di analisi, i processi con $\text{UID} \geq 10000$ (UID delle applicazioni Android) che invocano `accept` vengono segnalati come anomali, poiché su Android i processi applicativi normalmente non agiscono da server.

Una caratteristica comune a tutte e quattro le probe di rete è la pulizia delle mappe temporanee nel blocco `END`, necessaria per evitare che le strutture dati in memoria nel kernel rimangano allocate al termine del programma `bpfttrace`.

3.4 L'orchestratore: monitor.py

`monitor.py` è il componente che coordina l'intero ciclo di vita di una sessione di monitoraggio. All'avvio legge la mappa delle probe dal file `config/probes_map.json`, verifica che la probe richiesta sia presente nel registro, crea una directory di sessione sotto `sessions/` con nome composto dal codice della probe e dal timestamp corrente (ad esempio `syscalls_2025-01-15_10-30-00`), e salva immediatamente un file `session.json` con i metadati della sessione.

3.4.1 Gestione del processo bpfttrace

`bpfttrace` viene avviato come sottoprocesso tramite `subprocess.Popen` con `stdout` e `stderr` come pipe separate, in modalità line-buffered. Questa separazione è essen-

ziale: `stdout` trasporta esclusivamente gli eventi JSON emessi dalle probe tramite `printf`, mentre `stderr` contiene i messaggi diagnostici di `bpfttrace` (errori di compilazione, avvisi sui tracepoint, messaggi di avvio). Un thread daemon separato drena continuamente `stderr` e lo scrive su un file `stderr.log` all'interno della directory di sessione, garantendo che il thread principale non si blocchi mai in attesa di dati su `stderr`.

3.4.2 Elaborazione degli eventi

Il thread principale itera riga per riga sullo `stdout` di `bpfttrace`. Per ogni riga non vuota tenta il parsing JSON; le righe che non sono JSON valido vengono considerate output diagnostico e reindirizzate su `stderr.log`. Gli eventi JSON validi vengono processati attraverso una pipeline di tre fasi:

Normalizzazione (`normalize_event`): garantisce che ogni evento rispetti lo schema minimo atteso. Se il campo `schema_version` manca, viene aggiunto con il valore definito nella configurazione della probe. Se i campi `type` o `event` mancano, vengono aggiunti come fallback dalla configurazione. Se il campo `data` non è presente o non è un oggetto, viene inizializzato come dizionario vuoto. Viene inoltre aggiunto il campo `probe_code` per facilitare la correlazione fuori dalla cartella di sessione.

Arricchimento (`enrich_exec_event`): attivo solo per gli eventi di tipo `exec`. Legge `/proc/<pid>/cmdline` per ottenere la riga di comando completa del processo appena eseguito (i valori sono separati da caratteri null, che vengono sostituiti con spazi). Legge inoltre il link simbolico `/proc/<pid>/exe` per ottenere il percorso assoluto dell'eseguibile. Questo arricchimento compensa il limite di 15 caratteri del campo `comm` nel kernel e aggiunge informazioni non accessibili direttamente da `bpfttrace` in un tracepoint.

Validazione (`validate_event`): confronta i campi `type` ed `event` dell'evento ricevuto con i valori attesi dalla configurazione della probe. Le discrepanze generano avvisi su `stderr.log` ma non bloccano la scrittura dell'evento, evitando perdita di dati a fronte di probe multi-evento (indicate con `event: "*" nella configurazione`).

Ogni evento normalizzato, arricchito e validato viene serializzato nuovamente in JSON e scritto su `events.jsonl` con flush immediato, garantendo che i dati

vengano persistiti su disco in tempo reale anche in caso di interruzione improvvisa del processo.

3.4.3 Gestione della sessione e terminazione

Alla ricezione del segnale di interruzione (Ctrl-C, gestito tramite un blocco `try/except` su `KeyboardInterrupt`), `monitor.py` esegue la pulizia ordinata: termina il processo `bpfftrace` tramite `SIGTERM` seguito da `SIGKILL`, aggiorna il file `session.json` con il timestamp di fine sessione e stampa un riepilogo a terminale. La directory di sessione finale contiene quindi: `events.jsonl` (gli eventi raccolti), `stderr.log` (output diagnostico di `bpfftrace`) e `session.json` (metadati con orari di inizio e fine).

3.5 Il launcher interattivo: `launcher.sh`

`launcher.sh` è lo script `bash` che costituisce il punto di ingresso principale per l'utente. All'avvio esegue una serie di controlli preliminari verificando la presenza di `python3`, `bpfftrace`, `jq` e `timeout` nel `PATH`, e la disponibilità dei file `monitor.py` e `config/probes_map.json`. In caso di errore viene mostrato un messaggio diagnostico chiaro e lo script termina con codice di uscita 1.

La lista delle probe disponibili viene costruita dinamicamente leggendo le chiavi di `probes_map.json` tramite `jq`, e filtrata tenendo solo le probe il cui file `.bt` esiste effettivamente su disco. Questa lettura dinamica garantisce che il menu sia sempre sincronizzato con lo stato reale del filesystem, senza necessità di aggiornamenti manuali.

3.5.1 Modalità operative

Lo script supporta tre modalità operative accessibili dal menu principale:

Avvio probe: permette di selezionare una o più probe (con selezione multipla per numero o `A` per tutte) e un timer in secondi. Le probe vengono eseguite sequenzialmente, ognuna tramite il comando `timeout` che invia `SIGTERM` a

`monitor.py` allo scadere del tempo. Durante l'esecuzione viene mostrata una barra di avanzamento ASCII animata che aggiorna stato e percentuale ogni secondo.

Generazione report: mostra una lista delle sessioni disponibili in `sessions/`, con il numero di eventi raccolti e il timestamp di avvio (letto da `session.json`). L'utente sceglie la sessione e lo script invoca `summary.py` per generare il report.

Full monitoring: combina avvio probe e generazione report in sequenza automatica. Dopo la conclusione di ogni probe, il launcher localizza la directory di sessione appena creata estraendo il percorso dall'output di `monitor.py` tramite `grep`, con un fallback che cerca la directory più recente in `sessions/`, e invoca immediatamente `summary.py` su di essa.

La variabile d'ambiente `PYTHONUNBUFFERED=1` viene impostata prima di lanciare `monitor.py` per forzare lo svuotamento immediato del buffer stdout di Python, evitando che gli output vengano persi quando `timeout` termina il processo prima che i buffer vengano svuotati.

Lo script gestisce anche la modalità non interattiva: se viene passato il flag `-p` con il percorso di una probe, il menu viene saltato e la probe viene avviata direttamente. Se viene passato anche `-t`, il timer viene impostato al valore specificato invece di richiedere input dall'utente.

3.6 Il modulo di analisi: `summary.py`

`summary.py` è il componente responsabile dell'analisi post-sessione. Accetta come argomento il percorso di una directory di sessione, legge `events.jsonl` e `session.json`, calcola un insieme di metriche aggregate e produce sia un report testuale che un file `index.json` con tutti i dati analitici in forma strutturata.

3.6.1 Analisi generale degli eventi

La prima fase di analisi calcola statistiche generali: il numero totale di eventi, la loro distribuzione per tipo (`syscall`, `process`, `ipc`, `network`) e per nome specifico (`execve`, `openat`, `fork`, `binder_transaction`, ecc.), e i processi più attivi per volume di eventi generati. Viene inoltre calcolato il tasso di eventi al secondo, sia

tramite i metadati di sessione (`started_at` e `stopped_at` da `session.json`) sia tramite il campo `ts_ns` degli eventi stessi, che permette di identificare il picco di attività al secondo e la finestra temporale più intensa.

3.6.2 Analisi delle syscall e della latenza

Per gli eventi di tipo `syscall` vengono calcolati conteggi per nome, numero di errori (rilevati tramite valore di ritorno negativo) e tasso di errore percentuale. Se gli eventi includono il campo `lat_us` (latenza in microsecondi, presente nelle probe che correlano `sys_enter` e `sys_exit`), vengono calcolati i percentili p50, p95 e p99 sia globalmente che per nome di syscall e per processo. I 20 eventi più lenti vengono elencati separatamente, e viene prodotta una distribuzione dei codici errno più frequenti, sia globale che per singola syscall.

3.6.3 Analisi del Binder IPC

L'analisi Binder richiede la correlazione di tre tipi di eventi distinti. `summary.py` costruisce due indici in memoria: uno indicizzato per `debug_id` sugli eventi `binder_transaction_alloc` (per recuperare la dimensione del payload) e uno sugli eventi `binder_transaction_received` (per recuperare il `comm` del processo destinatario). Per ogni evento `binder_transaction` non di tipo `reply`, viene cercata la corrispondenza nei due indici, costruendo così una visione completa della transazione con mittente, destinatario, dimensione dati e tipo (one-way vs sincro).

Dal grafo delle transazioni viene costruito un IPC graph che mappa gli archi di comunicazione tra processi, con il numero di chiamate e i byte trasferiti per ogni coppia mittente-destinatario. Questo grafo è il punto di partenza per analisi comportamentali: pattern insoliti di comunicazione, processi che si scambiano volumi anomali di dati, o servizi di sistema contattati da applicazioni non attese.

3.6.4 Analisi di rete

L'analisi di rete aggrega i dati delle quattro probe in un quadro coerente. Vengono costruiti: il port landscape (elenco delle porte effettivamente aperte, filtrato sui bind

con `ret == 0`), la lista dei processi che hanno accettato connessioni in ingresso, il volume di dati inviati e ricevuti per processo, e una timeline al secondo dei byte trasmessi. La timeline viene utilizzata per identificare il picco di invio e il picco di ricezione. I processi con $\text{UID} \geq 10000$ che agiscono da server tramite `accept` vengono segnalati separatamente come potenzialmente anomali, poiché su Android le applicazioni normalmente non aprono porte in ascolto.

3.6.5 Albero dei processi e resource map

A partire dai campi `pid` e `ppid` presenti in tutti gli eventi, `summary.py` ricostruisce l'albero genealogico dei processi osservati durante la sessione. La struttura ad albero viene serializzata sia in forma gerarchica (per la visualizzazione) che in forma flat (per la ricerca rapida). La resource map aggrega invece, per ogni processo, l'insieme dei file aperti (da eventi `openat`), degli indirizzi di rete contattati (da eventi `connect`) e dei binari eseguiti (da eventi `execve`), fornendo una sorta di fingerprint comportamentale per ciascun processo osservato.

3.6.6 Output

`summary.py` produce tre output distinti. Il file `index.json` nella directory di sessione contiene tutti i dati analitici in forma strutturata e machine-readable, pensato per elaborazioni successive o visualizzazioni esterne. Il report testuale, stampato su stdout e salvato in `reports/summaries/` con il nome della sessione come identificativo, presenta le stesse informazioni in forma leggibile, con sezioni separate per ogni categoria di analisi e una barra ASCII per la timeline di rete. Il formato del file salvato può essere plain text (`.txt`) o Markdown (`.md`), configurabile tramite il flag `-format`.

3.7 Il formato dei dati: JSON Lines

Tutti gli eventi prodotti dal sistema vengono persistiti [10] in formato JSON Lines [11] (`.jsonl`), dove ogni riga è un oggetto JSON indipendente e completo. Que-

sto formato è stato scelto per diverse ragioni pratiche: supporta il parsing incrementale riga per riga senza necessità di caricare l'intero file in memoria, è compatibile con strumenti di analisi standard come `jq` [7], `pandas` [20] e `Apache Spark` [3], e permette lo streaming in tempo reale poiché ogni evento è autonomo e non dipende dal contesto degli eventi precedenti.

Lo schema comune a tutti gli eventi include i campi di intestazione `ts_ns`, `ts`, `type`, `event`, `pid`, `ppid`, `tid`, `uid`, `comm`, e un oggetto `data` con i campi specifici della probe. Il campo `ts_ns`, espresso in nanosecondi di uptime del kernel, garantisce una risoluzione temporale sufficiente per la correlazione di eventi rapidi come le transazioni Binder. Il campo `ts`, invece, fornisce un orario leggibile in formato `HH:MM:SS`, utile per la visualizzazione nei report.

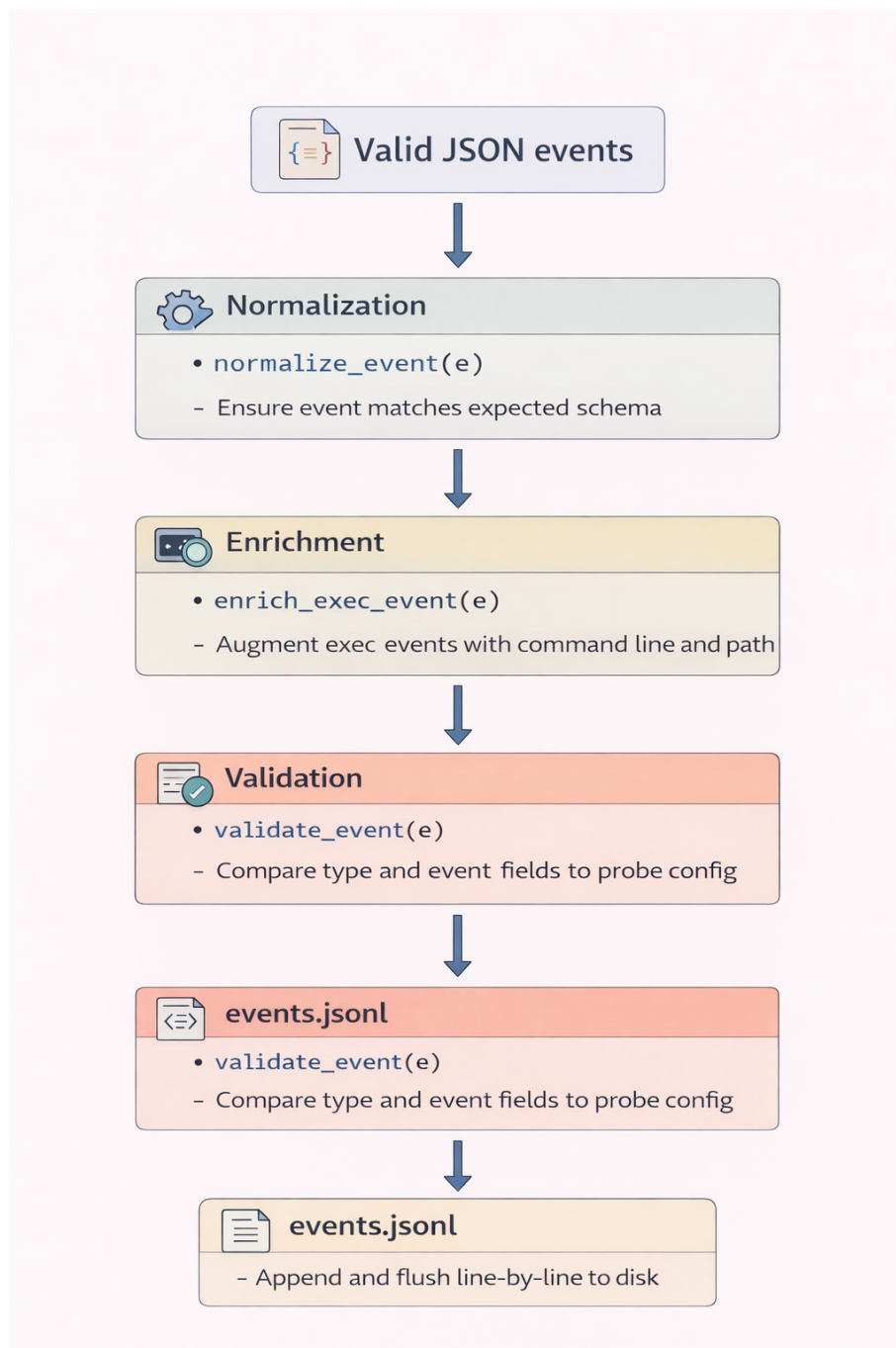


Figura 3.4: Pipeline di elaborazione degli eventi in `monitor.py`: normalizzazione, arricchimento e validazione.

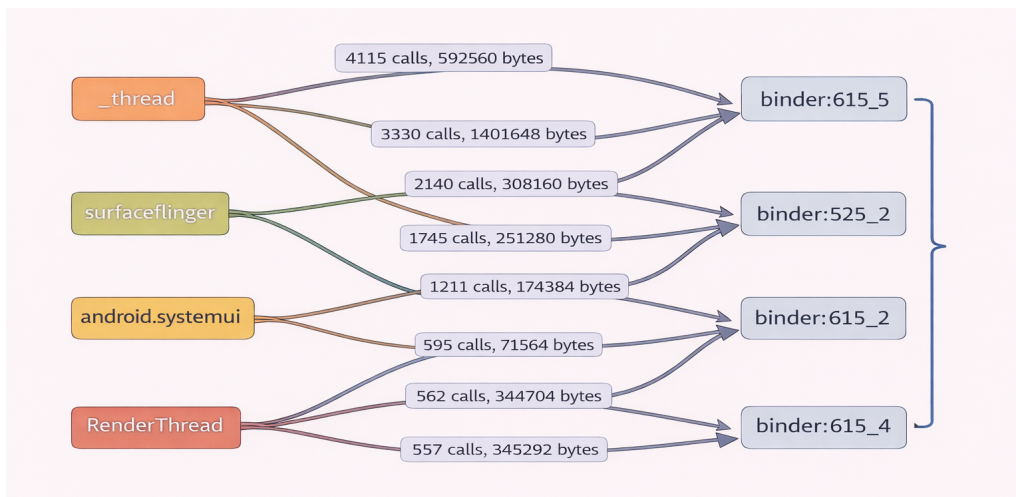


Figura 3.5: Esempio di output del report Binder IPC con grafo delle transazioni tra processi.

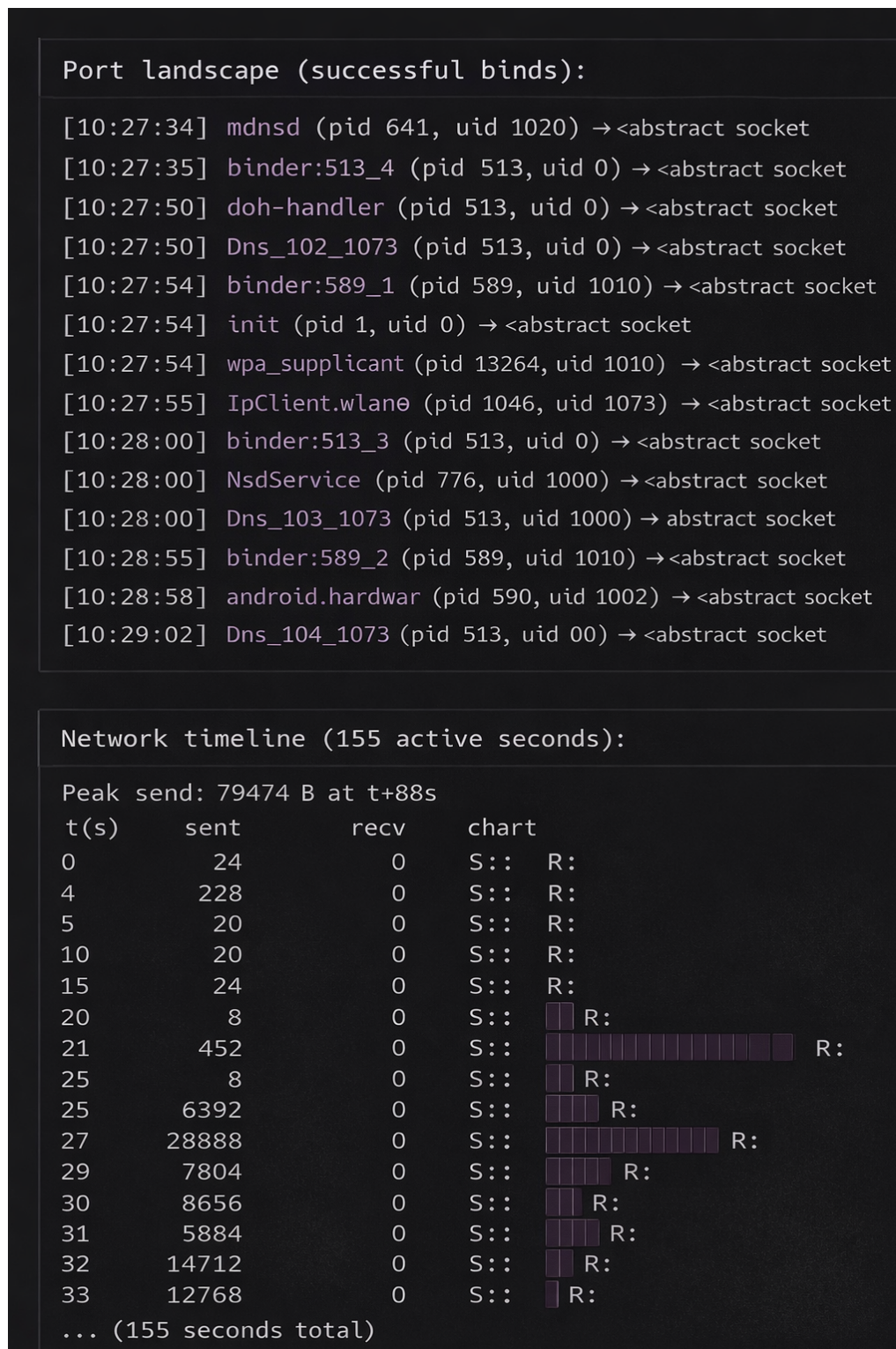


Figura 3.6: Esempio di output del report di analisi di rete con timeline del traffico e port landscape.

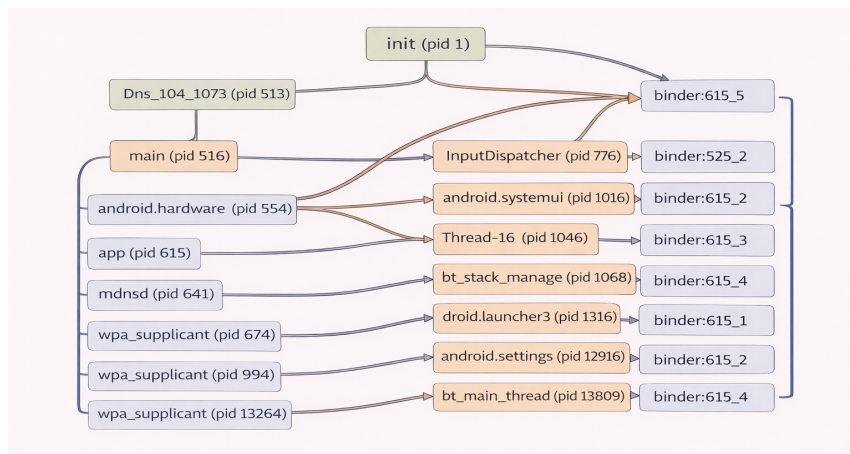


Figura 3.7: Esempio di albero dei processi ricostruito da summary.py a partire dai campi pid e ppid degli eventi.

Capitolo 4

Test e Validazione

In questo capitolo viene presentata l'analisi delle sessioni di monitoraggio raccolte sull'emulatore Cuttlefish durante scenari d'uso controllati. L'obiettivo è verificare la capacità del sistema di ricostruire il comportamento interno di Android a partire dagli eventi catturati dalle diverse probe, correlandone i risultati su più livelli dello stack software.

4.1 Attivazione della modalità aereo

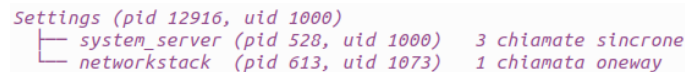
Il primo scenario analizzato riguarda l'attivazione della modalità aereo dall'applicazione Impostazioni (`com.android.settings`). L'interazione è avvenuta alle ore 10:28:55 durante una sessione di circa tre minuti; sette probe erano attive simultaneamente, ciascuna avviata in una sessione distinta identificata dal proprio codice: P00010 (syscalls), P00011 (Binder IPC), P00100 (process lifecycle), P01000 (net_bind), P01001 (net_accept), P01010 (net_send) e P01011 (net_recv).

Catena IPC Binder

L'analisi della sessione Binder (P00011) ha permesso di identificare il processo Settings tramite `adb shell dumpsys activity top`, che ha restituito il PID 12916 (uid 1000, comm `ndroid.settings`). Filtrando gli eventi nell'intervallo temporale del toggle, si osservano le prime transazioni significative esattamente alle 10:28:55: Settings invia tre chiamate sincrone (`oneway=0`) verso un thread di `system_server`

(pid 528, uid 1000) e una chiamata *fire-and-forget* (`oneway=1`) verso un thread Binder di `networkstack` (pid 613, uid 1073).

La catena Binder osservabile si esaurisce a questo punto: né `system_server` né `networkstack` generano ulteriori transazioni Binder in risposta al toggle. Questo comportamento è atteso: la gestione effettiva dell'interfaccia di rete avviene tramite chiamate al kernel attraverso socket `AF_NETLINK`, che sono invisibili alla probe Binder. La catena IPC ricostruita è la seguente:



```
Settings (pid 12916, uid 1000)
└─ system_server (pid 528, uid 1000)  3 chiamate sincrone
   └─ networkstack (pid 613, uid 1073) 1 chiamata oneway
```

Figura 4.1: Catena IPC Binder durante l'attivazione della modalità aereo.

Attività a livello di syscall

La sessione syscall (P00010), avviata sei secondi dopo la sessione Binder, copre ugualmente il momento del toggle. A partire dalle 10:28:57, il thread `IpClient.wlan0` (uid 1073) emette una serie di chiamate `connect()` su socket di famiglia `AF_NETLINK` (famiglia 16) e `AF_INET6` (famiglia 10), corrispondenti alle operazioni di smantellamento dell'interfaccia Wi-Fi a livello kernel. Alle 10:29:01 e 10:29:02 il volume di queste chiamate aumenta sensibilmente, coinvolgendo anche i thread `Thread-16` e `Thread-17` del processo `networkstack`.

Traffico di rete

L'analisi delle sessioni P01010 (`net_send`) e P01011 (`net_recv`) ha consentito di confrontare il traffico di rete prima e dopo il toggle. Nel periodo di baseline (10:28:00–10:28:50) si osserva che `IpClient.wlan0` effettua probe di connettività periodiche della dimensione di circa 58 KB ogni 46 secondi circa (rilevate alle 10:28:00 e 10:28:46), mentre `NsdService` mantiene un traffico costante di scoperta servizi sulla rete locale. Nessun traffico applicativo significativo è presente, coerentemente con il fatto che durante la sessione non era in corso alcuna navigazione.

Alle 10:29:01, circa sei secondi dopo il toggle, `IpClient.wlan0` riceve un ultimo burst di 58 392 byte: si tratta dell'ultima verifica di connettività prima che il client IP

riconosca la perdita dell'interfaccia. Contestualmente `NsdService` trasmette 29 364 byte, notificando la scomparsa dei servizi di rete precedentemente annunciati. Dopo le 10:29:02 il traffico proveniente dall'uid 1073 cessa completamente.

Ciclo di vita dei processi

La sessione P00100 (process lifecycle) ha rivelato un'attività inaspettatamente ricca nel momento del toggle. Alle 10:28:55, in coincidenza esatta con le transazioni Binder, `system_server` (pid 776) termina sei thread del pool (`Thread-57` fino a `Thread-62`) e ne crea altrettanti di nuovi (`child_pid` 13802–13807): il pool di thread viene ricreato in risposta al cambiamento di stato. Nello stesso istante esce il thread `handleEnable`, il cui nome indica esplicitamente il completamento dell'operazione di abilitazione; anche `servicemanager` genera un fork.

Alle 10:28:57 `networkstack` (pid 1046) crea un thread figlio tramite un proprio Binder thread. Tra le 10:29:02 e le 10:29:03 si osserva la fase conclusiva: `NetworkMonitor` spawna due processi figli per verificare lo stato della connettività, mentre i thread `Thread-15`, `Thread-16` e `Thread-17` di `networkstack` escono e vengono immediatamente ricreati, segnalando un riavvio interno del sottosistema di gestione della rete.

Sintesi della catena di eventi

I dati raccolti dalle diverse probe delineano una sequenza causale coerente che si estende su più livelli dello stack software:

Un'osservazione rilevante emersa dall'analisi è che nessuna singola probe è sufficiente a ricostruire l'intera catena. La probe Binder cattura il comando iniziale e i suoi destinatari diretti, ma non vede le operazioni kernel successive. La probe `syscall` mostra il punto di contatto con il kernel, ma non il contesto IPC che ha originato le chiamate. Le probe di rete e di lifecycle completano il quadro aggiungendo rispettivamente la conferma dell'interruzione del traffico e la riorganizzazione interna dei processi coinvolti. La correlazione temporale tra sessioni distinte si è rivelata quindi lo strumento fondamentale per la ricostruzione del comportamento del sistema.


```

10:28:55 Settings invia il comando via Binder
        → system_server: 3 chiamate sincrone
        → pool di thread ricreato
        → handleEnable termina
        → networkstack: 1 chiamata oneway

10:28:57 networkstack spawna un thread figlio
        IpClient.wlan0 avvia connect() su AF_NETLINK
        (smantellamento interfaccia a livello kernel)

10:29:01 IpClient.wlan0: ultima probe di connettività (~58 KB)
        NsdService notifica la perdita della rete (29 KB)

10:29:02 NetworkMonitor spawna figli (verifica stato rete)
        Thread-15/16/17 di networkstack escono e vengono ricreati
        UEventObserver riceve notifica kernel sull'interfaccia

10:29:03 Thread-15/16/17 definitivamente terminati
        Traffico uid 1073 assente da questo momento

```

Figura 4.2: Sequenza temporale degli eventi osservati durante l'attivazione della modalità aereo.

4.2 Navigazione web con Chrome

Il secondo scenario analizzato riguarda una sessione di navigazione web tramite il browser Chrome (`org.chromium.webview_shell`, uid 10064). La sessione, della durata di circa tre minuti, è stata registrata con le stesse sette probe del primo scenario. Le interazioni eseguite, con i rispettivi istanti approssimati, sono le seguenti: apertura della barra di ricerca a 10:07:08, ricerca di `google.com` a 10:07:15, clic sul risultato a 10:07:30, ricerca di `google news` a 10:07:48 e apertura di `YouTube` a 10:08:40.

Attività a livello di syscall

L'analisi della sessione syscall (P00010) ha messo in evidenza due tipologie distinte di eventi: le chiamate `connect()` emesse da `NetworkService` e le chiamate `openat()` distribuite su tutti i thread del processo.

Per quanto riguarda le `connect()`, la probe cattura il numero di connessioni aperte al secondo ma non gli indirizzi di destinazione: il campo `arg1`, che contiene il puntatore alla struttura `sockaddr` in memoria, non è decodificabile a posteriori dai log. Questa limitazione, intrinseca alla modalità di campionamento della probe,

impedisce la ricostruzione degli IP contattati e rappresenta un margine di miglioramento per versioni future del sistema. Nonostante ciò, i conteggi volumetrici mostrano picchi chiaramente correlati alle interazioni: 3 connessioni al secondo durante la navigazione di base, 39 al secondo al momento del caricamento di `google.com` (10:07:27) e un picco massimo di 84 connessioni al secondo a 10:07:50, corrispondente al caricamento della homepage di Google con tutte le sue risorse statiche. Per YouTube il picco è più contenuto (19 connessioni a 10:08:42), poiché il browser riutilizza le connessioni HTTP/2 già aperte verso i server Google anziché aprirne di nuove.

L'analisi delle `openat()` ha rivelato pattern di accesso ai file precisi e interpretabili. Nella fase di avvio (10:07:12–10:07:16) Chrome accede a `/dev/binder`, `/dev/ashmem` e `/dev/__properties__` per l'inizializzazione IPC, e al file `last-exit-info` per verificare l'assenza di crash alla sessione precedente. A 10:07:19 `NetworkService` legge `pubkey_sha256_blocklist.txt`, la lista dei certificati revocati, prima di stabilire qualsiasi connessione di rete: si tratta di un controllo di sicurezza che avviene sistematicamente all'avvio del sottosistema di rete. A partire da 10:07:21 compare in modo persistente la directory `cache/WebView/Default/H`, la cache HTTP del browser, acceduta continuamente da `NetworkService` e `ThreadPoolForeg` per verificare la presenza di risorse già scaricate. Il picco di 328 aperture di file in un secondo da parte di `ThreadPoolForeg` a 10:07:51 corrisponde al caricamento massivo delle risorse della pagina Google. Infine, a 10:08:44 compare `/apex/com.android.conscrypt/cacerts/5acf81` la lettura del certificato TLS radice necessario per validare il certificato di YouTube, e a 10:08:49 si attiva `AudioThread` con accessi alla directory `Default/Vid`, segnalando l'inizializzazione del sottosistema video in preparazione alla riproduzione.

Architettura IPC interna di Chrome

La sessione Binder (P00011) ha evidenziato un'architettura interna a canali separati e specializzati che rimane costante per tutta la durata della sessione.

Il primo canale è quello di rendering grafico: `RenderThread` comunica esclusivamente con `binder:615_1` (il processo `surfaceflinger`) tramite chiamate `oneway`, ovvero fire-and-forget. La scelta del modo `oneway` è coerente con la natura del

rendering: il browser non ha bisogno di attendere la conferma del compositor per procedere con il frame successivo.

Il secondo canale è quello dei dati di rete: **NetworkService** comunica esclusivamente con **CrRendererMain** tramite chiamate sincrone. I picchi di transazioni su questo canale corrispondono con precisione alle interazioni registrate: 106 chiamate sincrone al secondo a 10:07:27 durante il caricamento di **google.com**, 145 a 10:07:53 durante il caricamento della pagina, 127 a 10:08:43 durante il caricamento di YouTube.

Il terzo canale, visibile solo nei momenti di digitazione, è quello della tastiera virtuale: **InputConnection** invia chiamate **oneway** verso **putmethod.latin** durante le ricerche (10:07:39–10:07:42 e 10:08:35–10:08:36), notificando l’input dell’utente alla tastiera senza attendere risposta.

Un evento isolato di particolare interesse si osserva a 10:08:48–10:08:49: **AudioThread** contatta **servicemanager** per ottenere un riferimento al servizio audio, poi stabilisce una connessione con **binder:668_4** e infine, a 10:08:59, comunica con **AudioOut_15**. Questa sequenza — discovery del servizio tramite **servicemanager**, acquisizione del canale, connessione al dispositivo di output — rappresenta la procedura standard di inizializzazione audio su Android ed è osservabile con precisione nella traccia Binder.

Traffico di rete

Le sessioni P01010 (**net_send**) e P01011 (**net_recv**) hanno confermato la separazione architetturale già osservata nella traccia Binder. **NetworkService** mostra un profilo di traffico asimmetrico: invia volumi significativi di dati (le richieste HTTP) e riceve pochissimo, nell’ordine di poche decine di byte per secondo. **m.webview_shell**, al contrario, invia poco e riceve costantemente tra i 2 e i 17 KB al secondo: questi byte non rappresentano traffico esterno ma dati trasferiti da **NetworkService** al processo principale tramite socket Unix locale.

Il picco più rilevante è di 73.719 byte inviati da **NetworkService** a 10:08:03, circa otto secondi dopo il clic su **google.com**: è il momento in cui vengono scaricati i bundle JavaScript più pesanti della pagina. A confronto, il picco al momento del clic stesso (10:07:50) è di 12.054 byte, corrispondente alle richieste iniziali verso i server

Google. Per YouTube i picchi a 10:08:42 (19.962 byte) e 10:08:43 (14.333 byte) sono più contenuti e distribuiti, coerentemente con l'uso di connessioni HTTP/2 preesistenti.

InsetsAnimation compare sporadicamente con byte in ricezione nei momenti di transizione UI (apertura della tastiera, animazione della barra di ricerca): si tratta del sottosistema Android che trasmette coordinate di layout aggiornate al processo Chrome tramite socket locale.

Probe net_accept: un risultato atteso

La sessione P01001 (net_accept) ha registrato una sola connessione in ingresso per Chrome nell'intera sessione: quella di **Chrome_DevTools** a 10:07:14, il canale di debugging interno che il browser apre su socket locale all'avvio. L'assenza di ulteriori connessioni accettate è un risultato atteso e metodologicamente significativo: Chrome opera esclusivamente come client, aprendo connessioni verso l'esterno tramite `connect()` e non accettandone dall'esterno tramite `accept()`. Questo comportamento è coerente con il modello di sicurezza di Android, che non prevede che le applicazioni esponano porte in ascolto accessibili dalla rete. La quasi-assenza di eventi nella sessione net_accept costituisce quindi una conferma del corretto funzionamento del browser, oltre che una dimostrazione della capacità del sistema di rilevare anomalie: qualsiasi applicazione che mostrasse un volume significativo di `accept()` inaspettati rappresenterebbe un segnale di allerta.

Sintesi

L'analisi della sessione di navigazione web ha prodotto un quadro coerente e complementare rispetto al primo scenario. Rispetto alla modalità aereo, dove la catena causale era lineare e relativamente breve, la navigazione web ha rivelato un'architettura interna molto più articolata, con canali IPC specializzati per funzione, accessi al filesystem strutturati per fasi e un traffico di rete distribuito su processi distinti con ruoli ben definiti.

La correlazione tra le quattro probe ha permesso di ricostruire il flusso completo di una ricerca web a diversi livelli: la probe syscall mostra le connessioni aperte e

i file acceduti, la probe Binder mostra la comunicazione tra i componenti interni del browser e tra il browser e il sistema operativo, le probe di rete mostrano i volumi di dati scambiati e la direzione del flusso. Nessuna di queste probe, presa singolarmente, sarebbe sufficiente a descrivere il comportamento del sistema in modo completo.

4.3 Apertura della galleria e acquisizione fotografica

Il terzo scenario analizzato riguarda una sessione che combina l'uso dell'applicazione galleria (`com.android.gallery3d`, uid 10066) con l'acquisizione di una fotografia tramite la fotocamera integrata. La registrazione, della durata di circa tre minuti, ha coinvolto l'intera suite di sette probe. Le interazioni eseguite, con i rispettivi istanti riferiti all'avvio della probe syscall (`P00010_2026-03-04_10-32-37`), sono le seguenti: apertura della galleria a +15 s (10:32:52), chiusura della galleria e apertura della fotocamera a +48 s (10:33:25), scatto della fotografia a +64 s (10:33:41), prima riapertura della galleria a +78 s (10:33:55) e seconda riapertura a +170 s (10:35:27). Vale la pena osservare che la probe Binder (`P00011`) è stata avviata cinque secondi più tardi rispetto alla probe syscall: la fase di inizializzazione della galleria è pertanto visibile in modo completo solo nella traccia syscall e lifecycle, mentre la traccia Binder è disponibile a partire da 10:32:52.

Ciclo di vita dei processi

La probe `P00100` ha catturato in modo preciso i tre cicli di vita distinti del processo `droid.gallery3d` e l'intero ciclo di vita della fotocamera, rivelando una caratteristica rilevante dell'architettura Android: ogni apertura di un'applicazione che era stata chiusa dal sistema produce un nuovo processo con un nuovo PID.

Alla prima apertura della galleria (10:32:46), il processo `main` (pid 516, lo Zygote di Android) effettua un fork che genera `droid.gallery3d` con pid 14805. Nei secondi immediatamente successivi il processo espande il proprio pool di thread con binder worker, `RenderThread`, `GLThread` 41 e thread-pool generici. A 10:33:11, nel

momento in cui l'utente chiude la galleria per aprire la fotocamera, si osserva un exit di massa: tutti i thread di pid 14805 terminano in rapida successione nello stesso secondo, un pattern che distingue una chiusura controllata da un crash.

Tra 10:33:10 e 10:33:11 compare brevemente il processo `.intentresolver` (pid 15043), che Android avvia come intermediario per risolvere l'intent di apertura della fotocamera prima di chiudere la galleria.

L'apertura della fotocamera a 10:33:26 genera il pattern di lifecycle più complesso dell'intera sessione. `android.camera2` (pid 1913) espande il proprio pool con decine di thread specializzati in pochi secondi: `SoundDecoder`, `NDK MediaCodec`, `fifo-pool-thread` (thread a priorità real-time per il processing dei frame), `CameraHandler`, `pool-10-thread` e `CameraDeviceExecutor`. Parallelamente, a 10:33:27–10:33:29, il provider HAL della fotocamera (`camera.provider`, pid 604) e il servizio di fotocamera (`cameraservice`, pid 663) espandono i propri binder pool con nuovi worker (`binder:604_x`, `binder:663_x`). A 10:33:38–10:33:39, dopo lo scatto, compare `rs.media.module` (pid 1510), il processo responsabile della scrittura del file JPEG nel MediaStore.

La chiusura della fotocamera avviene a 10:33:47–10:33:48 con un nuovo exit di massa: terminano `Cam0_ResultDisp`, `C3Dev-0-ReqQueue`, `C3Dev-0-Status`, `StreamBufMgr`, `EmulatedSensor` e `camera.provider`. Alla prima riapertura della galleria (10:33:51), lo Zygote genera un nuovo processo con pid 15495, distinto dal precedente pid 14805. L'identità dell'applicazione è la stessa ma l'istanza di processo è nuova, con stato interno reinizializzato. A 10:35:13 tutti i thread di pid 15495 terminano e a 10:35:17 la terza istanza della galleria (pid 16236) viene creata con lo stesso pattern.

Accessi al filesystem

L'analisi delle `openat()` catturate dalla probe P00010 per uid 10066 rivela tre fasi distinte che si ripetono, con variazioni, a ogni apertura della galleria.

Nella fase di inizializzazione (10:32:46–10:32:48 per la prima apertura; 10:33:51–10:33:52 e 10:35:17–10:35:19 per le successive), i thread principali accedono a `/dev/__properties__` per leggere le proprietà di sistema, a `/data/dalvik-cache/x86_64/product@app@Gallery2@Gallery` per caricare il bytecode compilato dell'applicazione e alle librerie native del sistema

grafico in `/apex/com.google.cf.vulkan/lib64/` e `/vendor/lib64/`. Il `RenderThread` accede alla cache shader di Skia in `/data/user_de/0/com.android.gallery3d/cache/com.android.skia.shaders_cache`, riutilizzando shader precompilati per accelerare il rendering.

Nella fase di scansione del contenuto multimediale, osservabile a 10:32:51 e ripetuta a ogni riapertura, `SharedPreferences` legge la configurazione salvata dell'applicazione da `/data/user/0/com.android.gallery3d/shared_prefs/` e `thread-pool-1` avvia la scansione della cache di miniature in `/storage/emulated/0/Android/data/com.android.gallery3d/cache/`. A 10:32:55, nel momento in cui la galleria diventa visiva, il thread principale accede a `/storage/emulated/0/Pictures/IMG_20260303_221740.jpg`, la foto più recente presente nel dispositivo prima della sessione, e al file `share_history.xml`, che registra la cronologia di condivisione per ordinare i suggerimenti di condivisione.

Dopo lo scatto (10:33:57–10:34:01), i thread del pool (`thread-pool-0`, `thread-pool-2`, `thread-pool-3`) aprono in sequenza i tre file presenti nella cartella `Pictures`: `IMG_20260304_113335.jpg`, `IMG_20260304_113339.jpg` e `IMG_20260304_113342.jpg`. Quest'ultimo è il file appena acquisito; la sua comparsa nella traccia a 10:34:01 documenta il momento esatto in cui il `MediaStore` ha completato la scrittura e la galleria ne ha rilevato la presenza tramite la notifica del `Content Provider`. A 10:34:11 e 10:34:34 compaiono `Thread-12` e `Thread-13` che accedono a `/storage/emulated/0/Android/data/com.android.gallery3d/cache/remote-thumbnails`, la cache delle miniature remote: si tratta del thread di generazione delle anteprime che processa le immagini in background.

Architettura IPC Binder

La traccia Binder (P00011) mostra per la galleria un'architettura IPC sostanzialmente più semplice rispetto a Chrome, dominata da due destinatari fissi.

Il primo e principale destinatario è `binder:615_4`, un worker thread di `surfaceflinger` (pid 615). Verso di esso convergono quasi tutte le transazioni oneway della galleria: il thread principale, `GLThread 41` e `RenderThread` inviano frame grafici al compositor in modo fire-and-forget per tutta la durata della sessione, con frequenze di 10–40 transazioni al secondo durante la navigazione tra le foto e picchi fino a 42 transazioni

al secondo a 10:34:32. La scelta del modo oneway per il canale verso surfaceflinger è identica a quella osservata in Chrome e riflette un comportamento di sistema: nessuna applicazione attende la conferma del compositor prima di procedere con il frame successivo.

Il secondo destinatario è `binder:776_5`, un worker di `system_server` (pid 776). Verso di esso vengono indirizzate le transazioni sincrone della galleria, ovvero le richieste che richiedono una risposta: accessi al ContentProvider multimediale, notifiche di visibilità della finestra e query al servizio di gestione delle attività (ActivityManagerService). Le transazioni sincrone verso `binder:776_5` si intensificano nei momenti di apertura e chiusura dell'applicazione e in corrispondenza delle interazioni dell'utente.

Tre momenti nella traccia Binder sono particolarmente significativi. Il primo è a 10:33:05–10:33:07, quando `GLThread 41` e `RenderThread` aprono transazioni sincrone verso `binder:528_2` e `binder:613_5`: si tratta rispettivamente di `servicemanager` e `audioserver`, contattati in preparazione alla chiusura dell'applicazione. Il secondo è a 10:33:51, la riapertura della galleria dopo lo scatto: `droid.gallery3d` invia 43 transazioni sincrone nello stesso secondo verso `servicemanager`, `binder:776_5` e `binder:613_5`, una densità che riflette la reinizializzazione completa di tutti i sottosistemi dell'applicazione nel nuovo processo pid 15495. Il terzo è a 10:33:51–10:33:54, quando `Thread-3` e `Thread-3` inviano transazioni verso `binder:1510_6`: si tratta di `rs.media.module`, il processo MediaStore che ha già scritto la foto e sta notificando la galleria della sua disponibilità.

Nella fase di consultazione delle foto (10:34:18–10:35:10), le transazioni si stabilizzano in un ritmo regolare di 10–20 oneway al secondo verso surfaceflinger alternate a gruppi periodici di 5–16 transazioni sincrone verso `system_server`, pattern coerente con lo scorrimento dell'utente tra le immagini.

Traffico di rete locale

Le probe P01010 e P01011 hanno registrato esclusivamente traffico su socket Unix locali, senza alcun byte verso indirizzi IP esterni. Questo risultato era atteso per un'applicazione di gestione delle fotografie locali e ne conferma il comportamento.

Il mittente dominante nella traccia `net_send` è `app` (pid 615, ovvero `surfaceflinger`) con 537.600 byte inviati in 2.400 chiamate, seguito da `appSf` con 193.536 byte e `InputDispatcher` con 151.280 byte. `droid.gallery3d` compare come mittente con soli 4.072 byte in 143 chiamate: la galleria si limita a inviare comandi al compositor grafico tramite Binder, delegando l'effettivo trasporto dei buffer grafici a `surfaceflinger` stesso.

Il ricevitore dominante è il processo `logcat` (pid 676) con 906.160 byte ricevuti, un volume che riflette la generazione di log di sistema durante l'intera sessione. `droid.gallery3d` riceve 219.976 byte in 987 chiamate, corrispondenti alle notifiche IPC e ai dati di layout trasmessi da `system_server` tramite socket Unix. I picchi di traffico a 10:33:55 (80.817 byte ricevuti, 74.660 byte inviati) corrispondono alla riapertura della galleria dopo lo scatto, quando il sistema trasmette all'applicazione l'insieme aggiornato dei metadati multimediali tramite il `ContentProvider`.

Probe `net_accept` e `net_bind`: due risultati attesi

La sessione P01000 (`net_bind`) ha registrato zero eventi per l'intera durata della sessione: la galleria non apre alcuna porta in ascolto. La sessione P01001 (`net_accept`) ha registrato 280 eventi totali, nessuno dei quali attribuibile a `droid.gallery3d`: gli accept osservati appartengono a `init` (248 accept, sistema di multiplexing dei socket di sistema), `binder:513_2` (`netd`, con 12 accept periodici ogni 60 secondi, a 10:33:02, 10:34:02 e 10:35:02, riconducibili a `keepalive` di rete), `prng_seeder` (10 accept) e `traced` (10 accept).

Come nel caso di Chrome, l'assenza di accept e bind da parte dell'applicazione è un risultato metodologicamente rilevante. Per la galleria il significato è ancor più netto: a differenza di un browser, che potrebbe in linea teorica avere ragioni per aprire porte locali (come il canale DevTools), un'applicazione di gestione di file fotografici non ha alcuna ragione per farlo. Il valore dimostrativo delle probe `net_bind` e `net_accept` in questo scenario consiste nell'attestare che il comportamento osservato corrisponde al comportamento atteso, e che il sistema di monitoraggio è in grado di rilevare immediatamente qualsiasi deviazione da questo profilo.

Sintesi

Lo scenario galleria e fotocamera ha evidenziato aspetti dell'architettura Android non osservabili nei due scenari precedenti. Il più rilevante è la gestione del ciclo di vita dei processi: ogni apertura della galleria dopo una chiusura genera un nuovo PID, e la fotocamera coordina un ecosistema di processi distinti (`android.camera2`, `camera.provider`, `cameraservice`, `rs.media.module`) che si espandono e contraggono dinamicamente nel corso della sessione. La probe P00100 ha permesso di datare con precisione al secondo ogni transizione: apertura, chiusura e handoff tra processi sono leggibili come sequenze di fork ed exit nella traccia lifecycle.

La correlazione tra probe syscall e probe lifecycle ha consentito di collegare la comparsa del file `IMG_20260304_113342.jpg` nella traccia `openat()` alla notifica ricevuta dalla galleria tramite `rs.media.module`, ricostruendo la catena causale dallo scatto alla visualizzazione in galleria. Anche in questo scenario, nessuna singola probe avrebbe fornito una visione completa: la traccia Binder mostra la comunicazione tra i componenti, la traccia syscall mostra gli accessi ai file, la traccia lifecycle documenta la sequenza di nascita e morte dei processi e le tracce di rete confermano l'assenza di comunicazione esterna, tutti elementi che insieme compongono un quadro coerente e verificabile del comportamento del sistema.

Capitolo 5

Limiti e possibili miglioramenti

Capitolo 6

Conclusioni

Bibliografia

- [1] Android Open Source Project. *Binder IPC*. 2024. URL: <https://source.android.com/docs/core/architecture/hidl/binder-ipc>.
- [2] Android Open Source Project. *Cuttlefish Virtual Android Devices*. 2024. URL: <https://source.android.com/docs/setup/create/cuttlefish>.
- [3] Apache Software Foundation. *Apache Spark*. 2024. URL: <https://spark.apache.org>.
- [4] Matt Bishop e Michael Dilger. «Checking for Race Conditions in File Accesses». In: *USENIX Computing Systems*. Vol. 9. 2. 1996.
- [5] bpftrace contributors. *bpftrace Reference Guide*. 2024. URL: https://github.com/bpftrace/bpftrace/blob/master/docs/reference_guide.md.
- [6] John Doe e John Smith. «Paper title». In: *Proceedings of the Big Conf*. Giu. 2017, pp. 185–200. DOI: <https://doi.org/doi/reference/number>.
- [7] Stephen Dolan. *jq — Command-line JSON processor*. 2024. URL: <https://jqlang.github.io/jq/>.
- [8] ebpf.io. *What is eBPF?* 2024. URL: <https://ebpf.io/what-is-ebpf/>.
- [9] Matt Fleming. *A thorough introduction to eBPF*. 2017. URL: <https://lwn.net/Articles/740157/>.
- [10] IBM. *What is data persistence?* 2024. URL: <https://www.ibm.com/topics/data-persistence>.
- [11] JSON Lines contributors. *JSON Lines*. 2024. URL: <https://jsonlines.org>.
- [12] Linux kernel contributors. *perf: Linux profiling with performance counters*. 2024. URL: <https://perf.wiki.kernel.org>.

- [13] Linux man-pages project. *ptrace(2) — Linux manual page*. 2024. URL: <https://man7.org/linux/man-pages/man2/ptrace.2.html>.
- [14] Eric S. Raymond. *The Art of Unix Programming*. Addison-Wesley, 2003. URL: <http://www.catb.org/esr/writings/taoup/>.
- [15] Steven Rostedt. *ftrace — Function Tracer*. 2024. URL: <https://www.kernel.org/doc/html/latest/trace/ftrace.html>.
- [16] StatCounter. *Mobile Operating System Market Share Worldwide*. 2024. URL: <https://gs.statcounter.com/os-market-share/mobile/worldwide>.
- [17] strace contributors. *strace — system call tracer*. 2024. URL: <https://strace.io>.
- [18] The Linux Kernel Organization. *Task Structure — The Linux Kernel documentation*. 2024. URL: <https://www.kernel.org/doc/html/latest/>.
- [19] The Linux Kernel Organization. *The Linux Kernel*. 2024. URL: <https://www.kernel.org>.
- [20] The pandas development team. *pandas — Python Data Analysis Library*. 2024. URL: <https://pandas.pydata.org>.